Module 9 — Securing AI APIs for Production AI Systems

Learning Objectives

By the end of this module, you will be able to:

- Identify the unique **security risks** associated with serving ML models, including model abuse and cost explosion.
- Differentiate between **Authentication** (identity verification) and **Authorization** (permission management).
- Implement **password hashing** using `passlib` and `bcrypt` to protect user credentials.
- Generate and verify **JSON Web Tokens (JWT)** for stateless, secure communication.
- Secure inference endpoints using **FastAPI dependencies** and the `OAuth2PasswordBearer` scheme.
- Protect proprietary model weights and expensive GPU resources from unauthorized access.

Security Concepts for AI Engineers

1. Why AI APIs MUST be Secured

In a research environment, you focus on accuracy; in production, your model is a financial and legal asset.

- **Model Abuse:** Unauthorized users might spam your `/predict` endpoint, leading to massive cloud computing bills, especially if the model runs on GPUs.
- **Data Leakage:** Without security, sensitive data sent for inference could be intercepted or accessed by malicious actors.
- **Auth vs. Authorization: Authentication** is the process of confirming the user is who they claim to be (e.g., logging in). **Authorization** determines what that user is allowed to do (e.g., a "Basic" user might only access a small model, while an "Admin" can access the heavy, expensive model).

2. Core Security Tools

- **API Keys:** Often used for simple machine-to-machine communication, though the sources emphasize more robust standards.
- **JWT (JSON Web Tokens):** A standardized way to transmit information as a JSON object that is digitally signed. Because they are signed, the server can verify the sender's identity without checking a database every time.
- **Token Expiration:** For security, tokens should only be valid for a short time (e.g., 15–30 minutes) to minimize the damage if a token is stolen.

3. Password Hashing

Never store passwords in plain text. Instead, use a "Hashing" algorithm to turn the password into a complex string that cannot be reversed. FastAPI recommends using the `passlib` library with the `bcrypt` backend.

Code Examples & Explanations

Example 1: Hashing and Verifying Passwords

This utility ensures that user credentials are encrypted before they ever hit your database.

```
from passlib.context import CryptContext #

# Configure the hashing algorithm
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto") #

class Hash():
    def bcrypt(password: str): #
        return pwd_context.hash(password) #

    def verify(hashed_password: str, plain_password: str): #
        return pwd_context.verify(plain_password, hashed_password) #
```

Line-by-Line Explanation:

- **`CryptContext`**: Initializes the security tool that handles the "math" of hashing.
- **`schemes=["bcrypt"]`**: Specifically uses the `bcrypt` algorithm, which is a production standard.
- **`hash()`**: Takes a raw password and returns a secure, encrypted version for storage.
- **`verify()`**: Compares a raw password attempt with the stored hash to see if they match.

Example 2: Creating a JWT (Access Token)

After a user logs in, we give them a "ticket" (the JWT) to use for future model requests.

```
from datetime import datetime, timedelta #
from jose import jwt #

# DO NOT hardcode these in a real production environment
SECRET_KEY = "your-very-secret-key" #
ALGORITHM = "HS256" #

def create_access_token(data: dict): #
    to_encode = data.copy() #
    expire = datetime.utcnow() + timedelta(minutes=30) #
    to_encode.update({"exp": expire}) #
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM) #
    return encoded_jwt #
```

Line-by-Line Explanation:

- **`SECRET_KEY`**: A unique string used to sign the token so no one can forge it.
- **`timedelta(minutes=30)`**: Sets the token to expire after 30 minutes for security.
- **`jwt.encode`**: Bundles the user data and expiration time into a signed, encoded string.

Example 3: The Secured `/predict` Endpoint

This demonstrates how to use **Dependency Injection** to prevent unauthorized users from accessing your model.

```
from fastapi import FastAPI, Depends, HTTPException, status #
from fastapi.security import OAuth2PasswordBearer #
```

```
app = FastAPI()

# 1. Define where the token should be fetched from
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login") #

@app.post("/predict")
def predict_model(token: str = Depends(oauth2_scheme)): #
    # (The token is automatically verified by a function here)
    if not token: #
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED) #

    return {"result": "Expensive GPU prediction successful"} #
```

Line-by-Line Explanation:

- **`OAuth2PasswordBearer`**: This tells FastAPI that the client must send a "Bearer Token" in the header to access this route.
- **`Depends(oauth2_scheme)`**: Before the function runs, FastAPI will look for the token. If it's missing, the request fails with a **401 Unauthorized** error before your model code is even triggered.

AI-Specific Production Scenarios

- **Protecting Proprietary Models:** If you have spent millions training a model, securing the API is the only way to prevent competitors from "stealing" your model by reverse-engineering your outputs through massive, unrestricted queries.
- **Limiting Expensive Inference:** GPU time is billed by the second. Security allows you to implement **Authorization** so that only "Premium" users can access high-latency deep learning models, while "Guest" users are restricted.
- **Stateless Scaling:** Because JWTs contain all the necessary information, your server doesn't need to check a database for every single prediction request, which is critical for high-performance AI services.

Common Security Mistakes AI Engineers Make

- **Storing Secrets in Code:** Hardcoding the `SECRET_KEY` in your `main.py` file. This should always be in an environment variable.
- **Infinite Token Expiration:** Creating tokens that never expire, allowing a stolen token to grant permanent access to your model.
- **Ignoring the "Lock":** Developing a model and adding the security "decorator" last. Security should be integrated as you build your routes.
- **Plain Text Passwords:** Assuming the database is safe and storing raw passwords, which leads to total system compromise if the database is leaked.

Assignment: Secure a Prediction Route

1. Create a FastAPI app with a **POST** endpoint at `/inference`.
2. Import `OAuth2PasswordBearer` and define the `oauth2_scheme` pointing to a `login` URL.
3. Add a dependency to your `/inference` route so it requires a token.

4. Run the app and visit `/docs`. Verify that the `/inference` route now has a **Lock Icon**.
5. Try to "Execute" the route without logging in and confirm you receive a **401 Unauthorized** error.

Mini Project: Secured Inference API

Goal: Build a "Gatekeeper" service for an expensive model.

- Implement a **Login** route that accepts a username and password, verifies them using `passlib`, and returns a JWT token.
- Create a **Pydantic Schema** for a feature vector input.
- Create a **POST** `/predict` route that:
  - Requires a valid JWT token via a dependency.
  - Logs the user's ID for usage tracking.
  - Performs a dummy calculation (simulating model inference).
  - Returns the result only to authenticated users.

Security Checklist: Is my AI API safe?

- [ ] Are all user passwords stored as **encrypted hashes**?
- [ ] Does the `/predict` route have a **Lock Icon** in the Swagger UI (`/docs`)?
- [ ] Is the `SECRET_KEY` kept private and not checked into version control?
- [ ] Do **Access Tokens** have a strict expiration time (e.g., 30 minutes)?
- [ ] Does the API return a **401 Unauthorized** status if no token is provided?
- [ ] Are **Response Models** used to ensure no internal technical data (like model logits) is leaked?